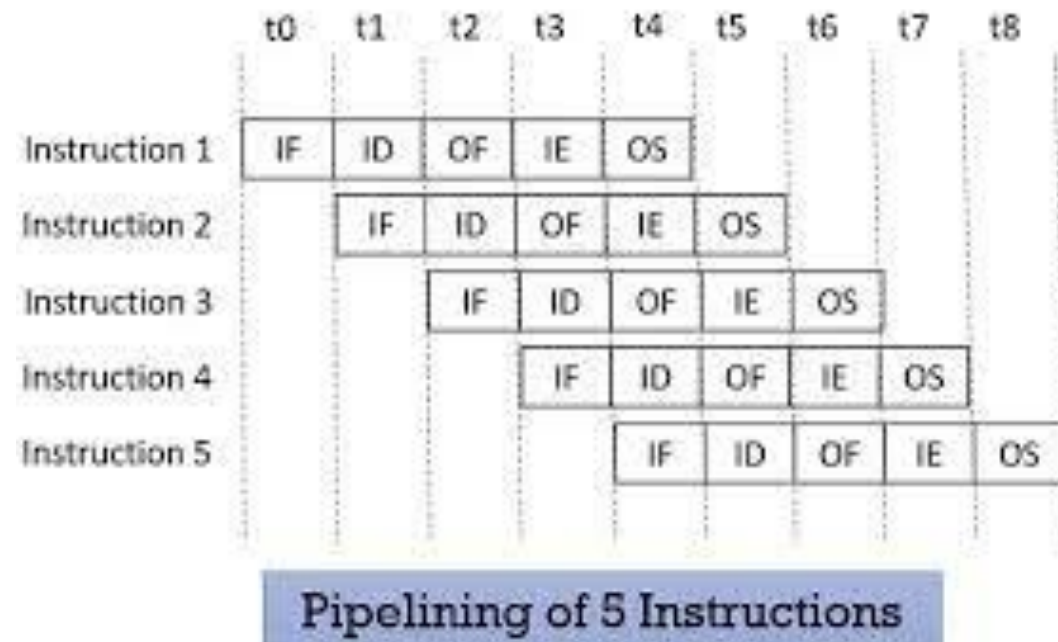


Pipelining

- **Pipelining** is a process of arrangement of hardware elements of the CPU such that its overall performance is increased.
- Simultaneous execution of more than one instruction takes place in a pipelined processor



- **Pipeline Stages** . 5 stages of the RISC pipeline with their respective operations:
- **Stage 1 (Instruction Fetch)** In this stage the CPU reads instructions from the address in the memory whose value is present in the program counter.
- **Stage 2 (Instruction Decode)** In this stage, instruction is decoded and the register file is accessed to get the values from the registers used in the instruction.
- **Stage 3 (Instruction Execute)** In this stage, ALU operations are performed.
- **Stage 4 (Memory Access)** In this stage, memory operands are read and written from/to the memory that is present in the instruction.
- **Stage 5 (Write Back)** In this stage, computed/fetched value is written back to the register present in the instructions.

Performance metrics of pipeline

❑ **Throughput:**

- It measure a number of instruction completed per unit time.
- , throughput number of instructions executed/ execution time.

❑ **Latency:**

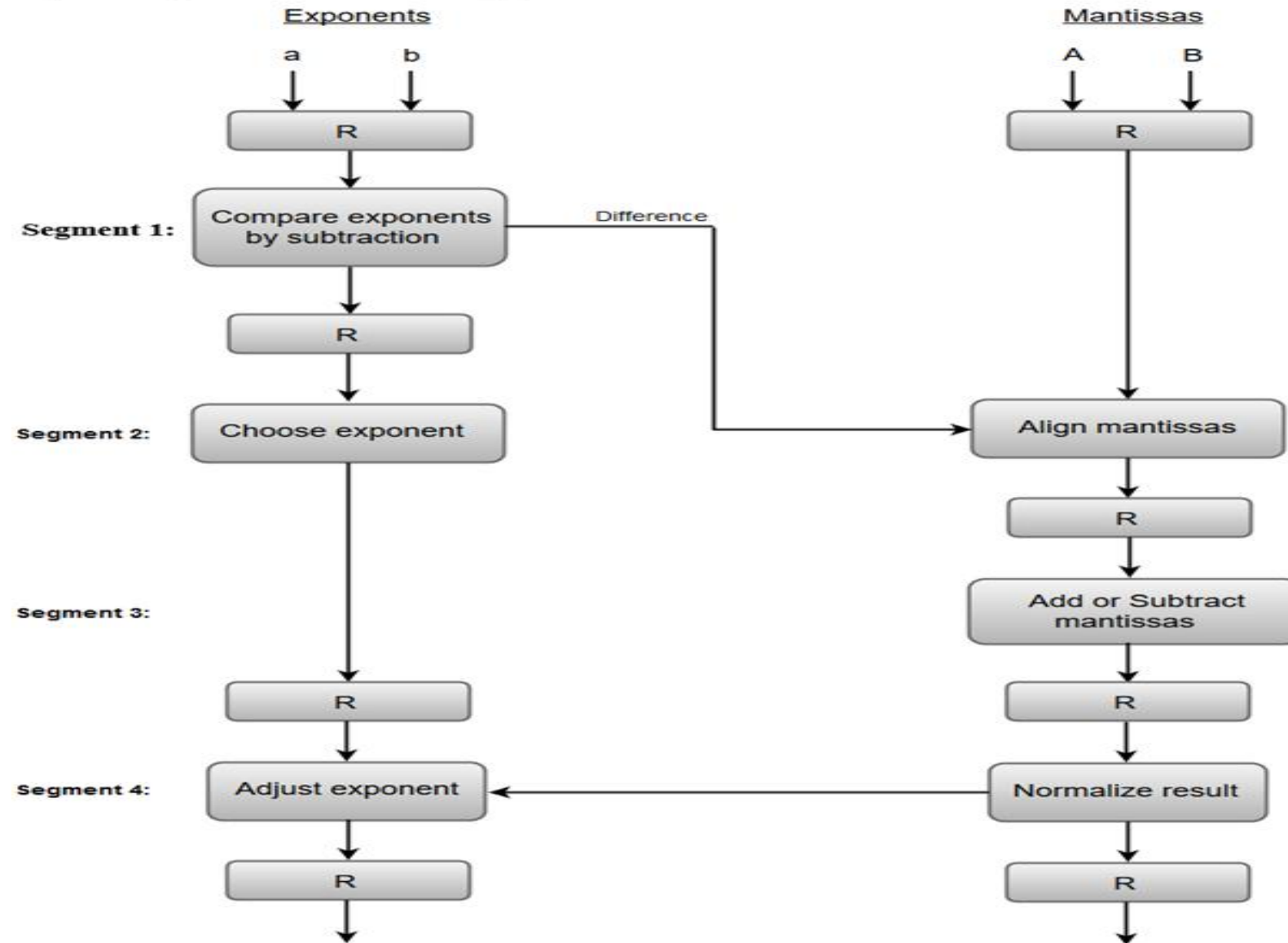
- It measure time taken for a single instruction to complete its execution.
- It is calculated as, $\text{Latency} = \frac{\text{Execution time}}{\text{Number of instructions executed}}$.
- It is influenced by pipeline length, depth, clock cycle time, instruction dependencies and pipeline hazards.

Arithmetic Pipeline

- Arithmetic Pipelines are mostly used in high-speed computers. They are used to implement floating-point operations, multiplication of fixed-point numbers, and similar computations encountered in scientific problems.
- $X = A * 2^a = 0.9504 * 10^3$
- $Y = B * 2^b = 0.8200 * 10^2$
- Where **A** and **B** are two fractions that represent the mantissa and **a** and **b** are the exponents.

- The suboperations that are shown in the four segments are:
- Compare the exponents by subtraction.
- Align the mantissas.
- Add or subtract the mantissas.
- Normalize the result.

Pipeline organization for floating point addition and subtraction:

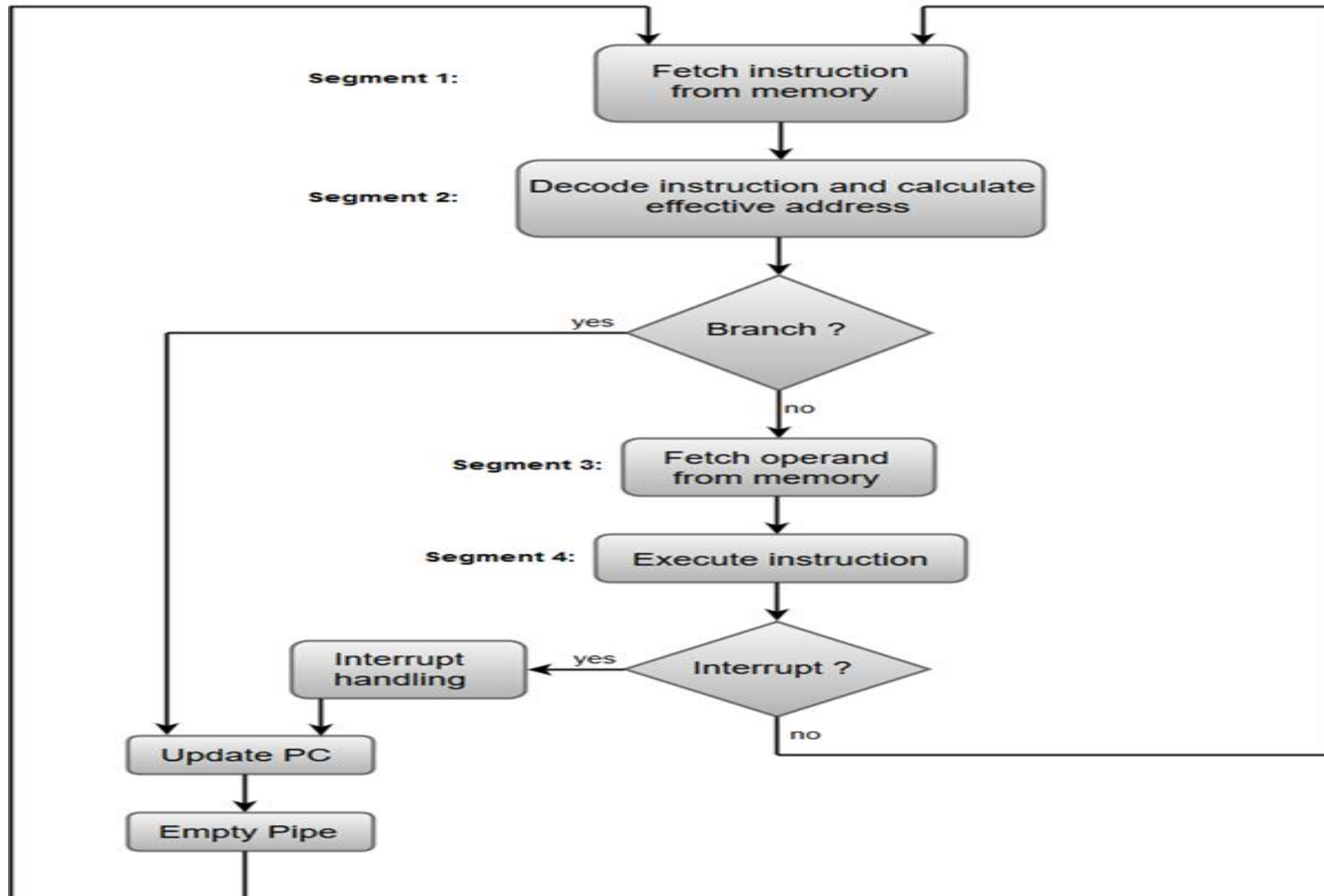


Instruction Pipeline

- Pipeline processing can occur not only in the data stream but in the instruction stream as well.
- Most of the digital computers with complex instructions require instruction pipeline to carry out operations like fetch, decode and execute instructions.

- the computer needs to process each instruction with the following sequence of steps.
- Fetch instruction from memory.
- Decode the instruction.
- Calculate the effective address.
- Fetch the operands from memory.
- Execute the instruction.
- Store the result in the proper place.

- A **four-segment instruction** pipeline combines two or more different segments and makes it as a single one. For instance, the decoding of the instruction can be combined with the calculation of the effective address into one segment.
- Segment 1:
- The instruction fetch segment can be implemented using first in, first out (FIFO) buffer.
- Segment 2:
- The instruction fetched from memory is decoded in the second segment, and eventually, the effective address is calculated in a separate arithmetic circuit.



- Segment 3:
- An operand from memory is fetched in the third segment.
- Segment 4:
- The instructions are finally executed in the last segment of the pipeline organization.

Pipeline hazards

- Pipeline hazards are conditions that can occur in a pipelined machine that impede the execution of a subsequent instruction in a particular cycle for a variety of reasons.
- Few instructions are at some stage of execution in a pipelined design. There is a chance that these sets of instructions will become dependent on one another, reducing the pipeline's pace.
- The dependencies in the pipeline are referred to as hazards since they put the execution at risk.

Types of Pipeline Hazards

- ☐ Structural
- ☐ Data
- ☐ Control

Structural Hazard

- Hardware resource conflicts among the instructions in the pipeline cause structural hazards. Memory, a GPR Register, or an ALU might all be used as resources here.
- When more than one instruction in the pipe requires access to the very same resource in the same clock cycle, a resource conflict is said to arise.
- In an overlapping pipelined execution, this is a circumstance where the hardware cannot handle all potential combinations.

Solution

- With the help of a hardware mechanism, we can minimize the structural dependency stalls in a pipeline. The mechanism is known as **renaming**.
- **Remaining:** In this mechanism, the memory will be divided into two independent modules, which are known as Data memory (DM) and Code memory (CM).
- Here, all the instructions are contained with the help of CM, and all the operands which are required for the instructions are contained by the DM.

Data Hazards

- Data hazards in pipelining emerge when the execution of one instruction is dependent on the results of another instruction that is still being processed in the pipeline.
- The order of the READ or WRITE operations on the register is used to classify data threats into three groups
- The data hazards are basically of three types:
 - RAW
 - WAR
 - WAW

Solution

- The WAR hazards and WAW hazards occur because the process contains a finite number of registers., these hazards are also known as the name dependencies.
- The processor will use the different registers to generate the output of each instruction if it contains an infinite number of registers. There is no chance of occurring the WAR and WAW hazards in this case.
- The WAR and WAW hazards will not cause the delay if a processor uses the same pipeline for all the instructions and executes these instructions in the same order in which they appear in the program. This is all because of the process of instructions flow through a pipeline.

Control Hazards

- Branch hazards are caused by branch instructions and are known as control hazards in computer architecture.
- The flow of program/instruction execution is controlled by branch instructions.
- Remember that conditional statements are used in higher-level languages for iterative loops and condition testing (correlate with while, for, and if case statements).
- These are converted into one of the BRANCH instruction variations. As a result, when the decision to execute one instruction is reliant on the result of another instruction, such as a conditional branch, which examines the condition's consequent value, a conditional hazard develops

Solution

- In the control dependency, we can eliminate the stall in the pipelines with the help of a method known as **Branch prediction**.
- The prediction about which branch will be taken is done at the 1st stage of branch prediction. The branch prediction contains the **0 branch penalty**.
- **Branch Penalty:** Branch penalty can be described as the number of stalls that are introduced at the time of branch operation in the pipelined.